

CS 101 Lab Quiz Practice

Balaji Karedla

September 10, 2024

1 Problems

1.1 Rock Paper Scissors

Rock-Paper-Scissors is a simple game played by two people, where the two players simultaneously gesture (or speak out) any one of the three objects 'rock', 'paper', or 'scissors'. The rules are straightforward: rock beats scissors, scissors beats paper, and paper beats rock. The outcome of each round is a win for the player whose object beats that of the other player, or a tie if both players choose the same object. For example, if player 1 chooses rock and player two chooses paper, then player 2 wins as paper beats rock.

Write a C++ program to accept two characters as input from the user. These denote the choice made by player 1 and player 2, respectively. The choices would be r for rock, p for paper, and s for scissors. Print which player wins (1 or 2, or T for tie) based on the character input.

1.2 Alternating Factorial Series Sum

You are given an infinite alternating series:

$$1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} \dots$$

Your task is to find the sum of the first N terms of this series. The series alternates in sign, with each term involving the reciprocal of a factorial.

1.3 Diamond Pattern

In this problem, you need to print two diamond-shaped patterns side by side, using asterisks (*) and spaces. They should have their middle rows aligned, separated by exactly three spaces between them.

A diamond's shape is determined by its size, which will be an odd integer. The size equals the number of rows needed to print the diamond; it also equals the number of columns (i.e., number of asterisks in the middle row). The sample given above shows two diamonds of size 5 and 3, side-by-side.

The main program is given to you that reads two integer values, 'n1' and 'n2', which represent the sizes of the two diamonds. It then calls a function printDiamond to print each row of each diamond. The function is given the size of the diamond to be printed, the height of the box in which it should be drawn (which is set to the maximum of n1 and n2), and an integer indicating which row in the box it should print. (Between the two calls to printDiamond for the same row, the main program prints 3 spaces and after the second call, it prints a newline character.)

Your task is to code the function printDiamond to correctly print the pattern of a single row of a single diamond, aligned in the middle of a box of a given height (you can assume that the box's height and the size of the diamond are both odd, and the box's height is at least as much as the size). Note that when the box's height is more than the size of the diamond, there will be empty lines (i.e., only spaces) at the top and bottom lines. Also note that every row should have exactly the same number of characters (including leading and trailing spaces), so that the second diamond is not distorted.

1.4 Matrix Identification

In the magical world of mathematics, some very special matrices have secret identities. (Below, recall that the main diagonal runs from the top-left to the bottom-right of the matrix):

1. **The Upper Triangular Matrix:** This one is like a superhero who refuses to be weighed down. It keeps all its heavy numbers (non-zero ones) above the diagonal, leaving everything below it as zero.
2. **The Lower Triangular Matrix:** The opposite of the upper one, this matrix likes to keep things grounded, with zeros flying high above the diagonal.
3. **The Diagonal Matrix:** All its non-zero entries are on the main diagonal, while the rest of the matrix is full of zeros, sneaking around unnoticed.
4. **The Identity Matrix:** The ultimate secret agent- sneaky like a diagonal matrix, but sophisticated with all ones on the main diagonal.
5. **The Zero Matrix:** A matrix representing pure silence-every element, from the top-left to the bottom-right, is zero. It's like the matrix decided to take a vow of silence and keep all its secrets hidden.

Write a C++ program that reads a matrix from the user and identifies its secret identity. The matrix is a square matrix of size 3×3 . The program should print the identity of the matrix based on the above definitions. It should print the most specific identity that the matrix satisfies. For example, if a matrix is both upper triangular and diagonal, it should be identified as diagonal.

1.5 Balanced Paranthesis

A sequence of characters '(' and ')' is said to be balanced or valid, if it can be obtained from valid mathematical expression by erasing all other symbols. For instance, a mathematical expression like $(1 + 2) * ((3 + 4) * (1 + 2))$ yields a balanced paranthesis sequence $()((()))$. On the other hand, a sequences like $()()$ or $()($ are not balanced.

Write a C++ program to accept a sequence of characters to identify whether the sequence is balanced or not.

1.6 Prime Factorization

Prime factorization of a positive number N writes it as a product of prime numbers, each raised to a positive integer exponent. For example, the prime factorization of 180 is $2^2 \times 3^2 \times 5^1$.

A **main** program is given to you that reads an integer N from the user, and prints out its prime factorization in the format $p_1^{n_1} \times p_2^{n_2} \times \dots \times p_k^{n_k}$ where $n_1, n_2, \dots, n_k$ are the exponents of k prime factors $p_1, \dots, p_k$. For example, on input 180, it prints out $2^2 \times 3^2 \times 5^1$. This program uses a function **extractFactor** to extract one prime factor and its exponent from the number passed to it; the extracted factor (prime number raised to the exponent) gets removed from the number, so that the main program can simply call this function in a loop and obtain the entire prime factorization.

Your task is to:

- Define a struct called **Factor** which can hold two integers – a prime number and an exponent (inspect the main program to see what the members of **Factor** should be called).
- Write a function called **extractFactor** that takes an integer N as an argument, and returns a **Factor** type value $\{p, n\}$ where p is the smallest prime factor of N and n is the largest integer such that p^n is a factor of N ; the function also removes the factor p^n from N . E.g., on passing $N = 180$ as the argument to **extractFactor**, it should return $\{2, 2\}$ (as a **Factor**) and update N to $180/2^2 = 45$.